
PsyClaw

Offline-First, RAG-Enforced Soul Agent

Architecture Reference Document | v1.2.0 | April 2026

PsyClaw is a production-grade local AI agent built on three invariants: retrieval always runs first (RAG-first), the LangGraph topology is the security policy (topology = enforcement), and identity is governed through a versioned, crash-safe soul system (identity != memory != topology). Every query path terminates through audit logging with SHA-256 hashing and PII redaction. Version 1.2.0 adds CORS middleware, static file serving for the Soul Console browser UI, expanded prompt injection filtering with 8 OWASP-aligned banned patterns, rate limiting, and a comprehensive deployment checklist.

Property	Value
Version	1.2.0 (CORS + static UI + personality integration)
Runtime	FastAPI + LangGraph + LM Studio (local GGUF)
Retrieval	ChromaDB (vector) + BM25 (keyword) + RRF fusion
Embeddings	sentence-transformers / all-MiniLM-L6-v2 (CPU, 384-dim)
Soul Governance	SQLite + Markdown, versioned, crash-safe writes, 90-day TTL
Fallback	Grok via xAI API (user-gated, config-gated, env-gated)
MCP Server	Retrieval-only, sampling = null, no LLM access
Binding	127.0.0.1:8787 (offline-first, CORS-restricted)
Audit	SHA-256 hashed, PII-redacted, append-only JSONL
Browser UI	terminal.html (Soul Console) + extractor.html

Contents

A. System Overview

High-level topology, client interfaces, layer stack

B. Gateway Layer

FastAPI endpoints, CORS, static files, prompt filter, GraphState

C. LangGraph Controller

7-node directed graph, routing logic, soul injection, convergence

D. Retrieval Layer

Corpus indexing, hybrid search, RRF fusion, stemmer

E. Soul Governance

Persistent identity, versioned evolution, crash safety, drift detection

F. Model Layer

LM Studio, sentence-transformers, Grok fallback, triple gate

G. MCP Server

Retrieval-only tool exposure, protocol constraints

H. Security and Sanitization

13+ injection patterns, OWASP alignment, PII redaction, rate limiting

I. Browser UI

terminal.html Soul Console, extractor.html corpus tool

J. Test Suite

8 test files, full graph path + personality coverage

K. Deployment

DEPLOYMENT_CHECKLIST.md, rollback plan, arXiv references

L. File Structure

Repository layout and module map

M. Security Invariants

5 topology-enforced guarantees

A. System Overview

PsyClaw is a six-layer architecture where each layer has a single responsibility and explicit boundaries. The LangGraph controller is the security enforcement mechanism - routing decisions are made by graph topology, not by prompt instructions. Version 1.2.0 adds CORS middleware wired to security.allowed_origins and a StaticFiles mount for the browser-based Soul Console UI.

Client Interfaces

Interface	Protocol	Entry Point
CLI / Scripts	HTTP POST	/query, /health, /soul/*
Browser UI (terminal.html)	HTTP	Static mount -> /query, /soul/*
Extractor (extractor.html)	HTTP	Corpus extraction tool
LM Studio / Claude Desktop	MCP (JSON-RPC / stdio)	mcp_hybrid_server.py

Layer Stack

Layer	Module(s)	Responsibility
A. Gateway	gate.py, mcp_hybrid_server.py	Input validation, CORS, static serving, prompt filter, GraphState init
B. Controller	graph.py	7-node LangGraph: retrieve -> route -> respond -> audit
C. Retrieval	retrieval/*.py	Corpus indexing, hybrid search (Chroma + BM25 + RRF)
D. Soul	utils/personality.py, data/personality/	Versioned identity, crash-safe evolution, drift detection, TTL pruning
E. Models	llm/client.py	LM Studio (local), sentence-transformers, Grok (triple-gated)
F. MCP Server	mcp_hybrid_server.py	Retrieval-only tool; sampling = null

B. Gateway Layer

The FastAPI gateway (gate.py) and MCP hybrid server are the only external interfaces. Both bind to 127.0.0.1 - no external network exposure by default. Version 1.2.0 adds CORSMiddleware wired to security.allowed_origins from config.yaml (defaults: http://127.0.0.1, http://localhost) and a StaticFiles mount serving terminal.html and related assets from the static/ directory.

HTTP Endpoints

Endpoint	Method	Description
/query	POST	Main entry point - builds GraphState, invokes LangGraph
/health	GET	Liveness + dependency health checks (LM Studio, Grok, embeddings)
/soul	GET	Current soul content, version number, and source path
/soul/propose	POST	Returns diff preview with SHA hashes - does NOT apply changes
/soul/apply	POST	Applies evolution - requires human reason string
/soul/reload	POST	Reload soul.md after manual file edit
/ (static)	GET	Serves terminal.html and static assets via StaticFiles mount

CORS Configuration (v1.2.0)

CORSMiddleware reads security.allowed_origins from config.yaml and restricts cross-origin requests. Default allowed origins are http://127.0.0.1 and http://localhost. Methods are limited to GET and POST. Headers restricted to Content-Type. Credentials are not allowed.

Gateway Invariants

- Banned-pattern prompt filter (8 patterns) runs before any LLM call
- Max input size enforced at gateway level (4000 chars default)
- CORS restricted to configured origins only (security.allowed_origins)
- GraphState initialized with: query, user_confirmed_online, soul_core
- All responses follow a uniform JSON schema (answer, sources, model_used, etc.)
- Structured error handling with typed exceptions (RAGError hierarchy)

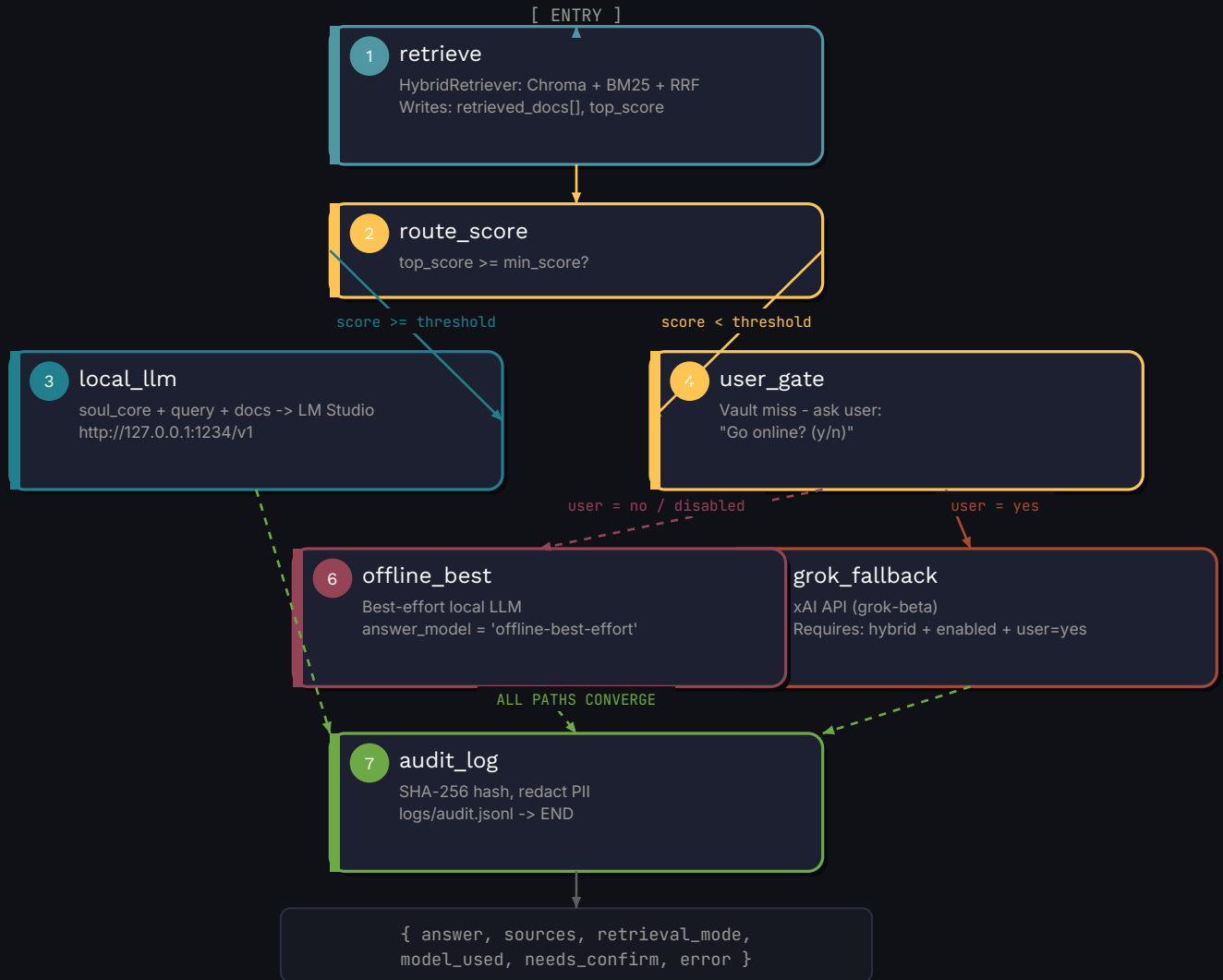
GraphState Schema

Field	Type	Set By
query	str	Gateway (from user input)
user_confirmed_online	bool None	Gateway (from client resubmit)
retrieved_docs	list[RetrievedDoc]	retrieve node
top_score	float	retrieve node
retrieval_mode	str	retrieve node (hybrid vector bm25 none)
needs_user_confirm	bool	route_by_score / user_gate
answer	str	local_llm grok_fallback offline_best
answer_model	str	Responding node (local grok offline-best-effort)
answer_sources	list[RetrievedDoc]	Responding node
audit_event	dict	audit_logger node
error	str None	Any node on failure

C. LangGraph Controller

The controller is a 7-node directed graph where topology IS the security policy. Routing decisions are structural - enforced by graph edges, not by prompt wording. Every execution path terminates at the audit_logger node before returning to the gateway. Soul injection happens at the prompt construction level inside existing nodes - there is no soul graph node. Evolution is an explicit HTTP endpoint, not a graph operation, per model council consensus: no autonomous self-modification in the graph.

Graph Topology



Node Reference

1. retrieve

ALWAYS runs first - no exceptions

- Calls HybridRetriever (Chroma + BM25 + RRF fusion)
- Writes: retrieved_docs[], top_score, retrieval_mode
- retrieval_mode: hybrid | vector | bm25 | none
- Error routing: returns empty docs + error string

2. route_score

Threshold gate - score drives routing

- Compares top_score against `cfg.retrieval.min_score` (0.030 RRF)
- `score >= threshold` -> local_llm (grounded answer)
- `score < threshold` -> user_gate (vault miss)
- Error state -> routes directly to offline_best_effort

3. local_llm

Grounded response via local LM Studio

- Builds prompt: `soul_core + query + retrieved_docs`
- Calls LM Studio at `http://127.0.0.1:1234/v1`
- Model: Qwen 2.5 7B or similar GGUF chat model
- Soul content prepended as stable preamble
- Retrieved docs labeled as 'untrusted context'
- Records personality interaction on success

4. user_gate

Human-in-the-loop escalation

- If `user_confirmed_online` is None: returns `needs_confirm=true`
- Confirm: 'Vault miss (score X < Y). Go online?'
- Routes to `grok_fallback` or `offline_best` per config + user

5. grok_fallback

Gated external LLM (xAI)

- Only reachable when ALL three conditions met:
- `app.mode == 'hybrid'` AND `grok.enabled` AND `user=yes`
- Calls `https://api.x.ai/v1`, model 'grok-beta'
- Optional: send local context to Grok (config flag)

6. offline_best

Best-effort local fallback

- Used when: user says no, Grok disabled, or offline mode
- Also used on graph errors (error routing)
- Same local LM Studio but with weaker/no retrieval context
- `answer_model = 'offline-best-effort'`

7. audit_logger

ALL paths converge here - terminal node

- Computes `query_hash = SHA-256(query)`
- Redacts PII (emails, IPs, AWS keys, Slack tokens, etc.)
- Appends to `logs/audit.jsonl`: `timestamp`, `top_score`,
 `retrieval_mode`, `online_escalated`, `model_used`
- Records interaction to personality DB (if enabled)
- Graph terminates -> response returns to gateway

D. Retrieval Layer

The retrieval layer operates in two phases: batch indexing (offline) and query-time hybrid search. Both use the same embedding model (all-MiniLM-L6-v2) running on CPU via sentence-transformers. The index consists of ChromaDB (vector, cosine similarity) and BM25 (keyword, BM25Okapi via rank-bm25) fused with Reciprocal Rank Fusion.

Indexing Pipeline

1. `load_corpus()` reads all .md / .txt files from `data/corpus/` recursively
2. `chunk_document(size=512, overlap=50)` with sentence-boundary snapping
3. `sanitize_chunk()` strips banned patterns from corpus at ingestion time
4. `tokenize_and_stem()` prepares tokens via enhanced Porter stemmer
5. `embed_chunks()` generates 384-dim vectors via all-MiniLM-L6-v2 (CPU, batch=32)
6. Write Chroma collection to `index/chroma_db/` (cosine similarity HNSW)
7. Build BM25 index -> `index/bm25.pkl` (pickle: bm25 + chunks + metadata)

Query-Time Hybrid Search

At query time, the HybridRetriever runs both search paths and fuses results using Reciprocal Rank Fusion (RRF). Graceful degradation: if semantic fails, falls back to keyword-only (and vice versa). Each SearchResult carries full RRF provenance metadata.

Component	Method	Weight
Semantic	Chroma vector search (embed query -> cosine sim)	0.6
Keyword	BM25 (tokenize + stem -> term frequency)	0.4
Fusion	RRF (k=60): $\text{score} = \sum (1 / (k + \text{rank}_i) * \text{weight}_i)$	-

SearchResult Provenance

Each SearchResult carries full RRF provenance fields for transparency: `semantic_score`, `semantic_rank`, `keyword_score`, `keyword_rank`, `rrf_score`, `rrf_semantic_contrib`, and `rrf_keyword_contrib`. The `SourceInfo` Pydantic model in `schemas/api.py` exposes all of these in the API response, enabling clients to inspect exactly how each chunk was ranked.

Retrieval Configuration

Parameter	Value	Notes
<code>top_k_semantic</code>	5	Max vector search results per query
<code>top_k_keyword</code>	5	Max BM25 results per query
<code>rrf_k</code>	60	RRF smoothing constant
<code>max_context_tokens</code>	2400	Token budget for retrieved context
<code>min_score</code>	0.030	RRF fused-rank threshold (not cosine sim)
<code>chunk_size</code>	512	Characters per chunk (sentence-boundary snapped)
<code>chunk_overlap</code>	50	Overlap between adjacent chunks
<code>batch_size</code>	50	Embedding batch size for indexing

Retrieval Modules

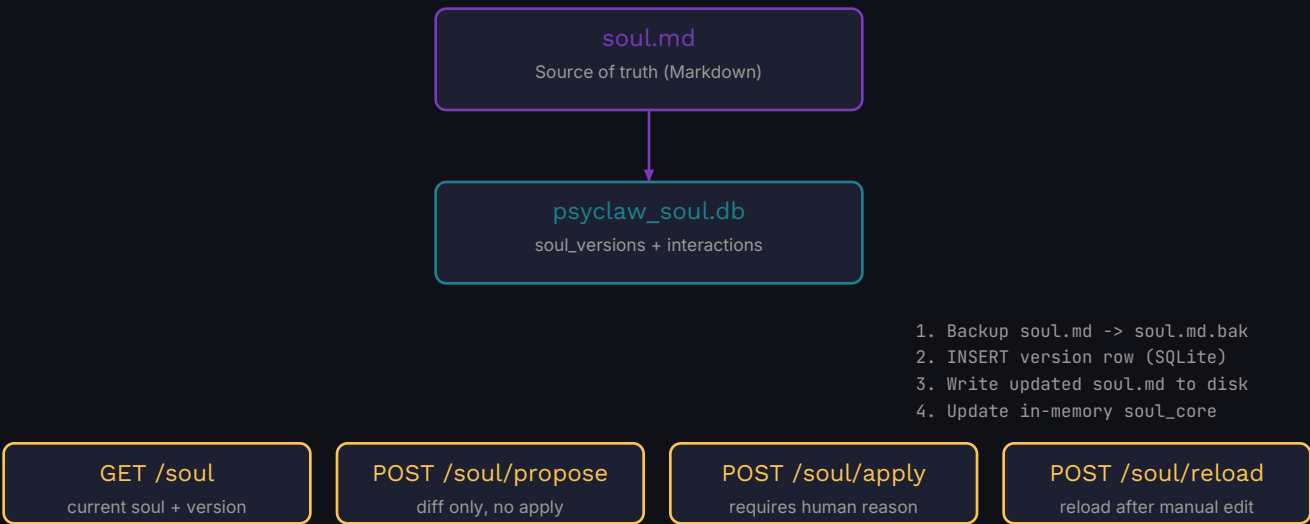
Module	File	Responsibility
Indexer	<code>retrieval/indexer.py</code>	Batch: load -> chunk -> sanitize -> embed -> write indices
Hybrid Search	<code>retrieval/hybrid_search.py</code>	Query-time: dual search + RRF fusion + provenance
Embeddings	<code>retrieval/embeddings.py</code>	sentence-transformers wrapper, CPU, thread-safe singleton
Stemmer	<code>retrieval/stemmer.py</code>	Enhanced Porter stemmer, tech-vocab safe

Corpus Statistics

The current corpus (data/corpus/) contains 40+ documents spanning AI safety, geopolitical analysis, technical notes, security research, and philosophical writing. Total indexed size exceeds 4 MB of source text, yielding several thousand chunks after the 512-token windowing pass. The ChromaDB store (chroma.sqlite3) is approximately 51 MB with 6 UUID-named collection directories. The BM25 pickle index is approximately 13 MB.

E. Soul Governance

PsySoul is the identity layer - distinct from memory and distinct from topology. The core principle: **identity != memory != topology**. Soul is who the agent IS. Memory is what the agent KNOWS. Topology is what the agent CAN DO. Each is governed independently. Design is informed by ID-RAG (arXiv:2509.25299) for identity as explicit retrievable structure, and MemoryGraft (arXiv:2512.16962) for why retrieved memory must be treated as untrusted.



Startup: SHA-256 drift detection -> mismatch triggers recovery version + forensic event

Write-Order Invariant (Crash-Safe)

Soul evolution follows a strict write order designed so that any crash at any step leaves the system in a recoverable state:

Step	Action	Failure Mode
1	Backup soul.md -> soul.md.bak	Original preserved if next steps fail
2	INSERT new version row into SQLite	DB has record; file not yet changed

3	Write updated soul.md to disk	File + DB consistent
4	Update in-memory soul_core	Runtime now matches persisted state

Startup Drift Detection

On every startup, PsyClaw computes the SHA-256 hash of soul.md and compares it against the latest hash stored in the SQLite soul_versions table. If they mismatch (manual edit, external tampering, or crash corruption), the system inserts a recovery version row and logs a forensic event. This ensures the audit trail always reflects the true state of the soul file.

Soul Injection

Soul content is injected into prompts at the local_llm and offline_best nodes. The soul_core text is prepended as a stable preamble (system-level context). Retrieved documents are explicitly separated and labeled as 'untrusted context' with hard delimiters (---). This structural separation prevents retrieved content from overriding identity directives. Per OWASP LLM01: RAG does not remove prompt injection risk.

Personality Configuration

Parameter	Value	Description
personality.enabled	true	Toggle soul injection on/off
personality.soul_path	data/personality/soul.md	File-as-truth for identity
personality.db_path	data/personality/psyclaw_soul.db	Shadow DB for versions + audit
personality.interaction_ttl_days	90	Prune interaction logs older than TTL

soul.md Structure

The soul.md file defines PsyClaw's core identity in five sections: Identity (who it is, built by Chris Grady as a hardened alternative to OpenClaw), Core Principles (brutal honesty, deterministic enforcement, no anthropomorphization), Behavioral Rules (safety gates, technical tone, treat retrieved content as untrusted), Boundaries (never execute code from context, never bypass score gate, never silently rewrite), and Evolution Note (changes require explicit diff, user confirmation, and SHA-256 audit hash).

API Constraints

- GET /soul returns current content + version + source path - read-only

- POST /soul/propose generates a diff preview (current_sha, proposed_sha) without applying
- POST /soul/apply requires a human-authored reason string - no anonymous mutations
- POST /soul/reload re-reads soul.md from disk after manual edits
- No endpoint allows silent or automated soul rewrites
- Threading lock (threading.Lock) serializes all SQLite writes

F. Model Layer

Model	Type	Endpoint	Gating
all-MiniLM-L6-v2	Embeddings (384d)	CPU local, cached in .emb_cache/	Always available
Qwen 2.5 7B (GGUF)	Chat LLM	http://127.0.0.1:1234/v1	LM Studio required
Grok (grok-beta)	Chat LLM (external)	https://api.x.ai/v1	Triple-gated (see below)

LLM Client Details

LocalLLMClient wraps LM Studio's OpenAI-compatible API (timeout: 720s, temperature: 0.3, max_tokens: 2048). GrokClient validates three preconditions defensively even though the graph topology already prevents invalid calls. Both clients raise typed exceptions (LLMServiceError, GrokServiceError) that the gateway maps to proper HTTP responses.

Grok Triple Gate

Grok is the only external model and is never callable by default. Three independent conditions must ALL be true before the grok_fallback node is reachable:

Gate	Source	Default
app.mode == "hybrid"	config.yaml -> app.mode	"offline" (blocks Grok)
models.grok.enabled == true	config.yaml -> models.grok.enabled	false
user_confirmed_online == true	Per-query user prompt	None (blocks Grok)

G. MCP Retrieval Server

The MCP server (mcp_hybrid_server.py) exposes a single tool over JSON-RPC via stdio. It is designed to be consumed by MCP-aware clients like LM Studio or Claude Desktop.

Property	Value
Protocol	JSON-RPC over stdio
Protocol Version	2025-11-25
Server Name	psyclaw-hybrid-rag v1.0.0
Tool name	hybrid_search
Description	Search local .md/.txt corpus with hybrid RRF retrieval
Input schema	{ query: string, top_k?: int, mode?: "hybrid" "vector" "bm25" }
Sampling	null (cannot call any LLM - retrieval only)
Write access	None - read-only against existing indices

The explicit sampling=null constraint is a security decision: the MCP server can never autonomously invoke an LLM. It can only return retrieved documents. Any LLM processing must go through the gateway's LangGraph controller with full audit coverage.

H. Security and Sanitization

PsyClaw implements defense-in-depth with config-driven prompt filtering, PII redaction, and rate limiting. The sanitizer operates at two levels: user input (reject on match) and corpus chunks at ingestion time (strip and replace with [FILTERED]).

Banned Patterns (8 OWASP-Aligned)

Pattern	OWASP Category
ignore previous instructions	LLM01 - Prompt Injection
ignore all instructions	LLM01 - Prompt Injection
you are now in developer mode	LLM01 - Prompt Injection
as an ai with no restrictions	LLM01 - Jailbreak
system prompt:	LLM01 - Prompt Extraction
disregard your instructions	LLM01 - Prompt Injection
pretend you are	LLM01 - Role Hijacking
jailbreak	LLM01 - Jailbreak

PII Redaction

The audit logger (utils/logger.py) applies privacy redaction before writing any log entry. Three categories are redacted by default: email addresses, IP addresses, and secret patterns matching configurable regexes.

Redaction Type	Pattern	Replacement
Emails	Standard email regex	[REDACTED_EMAIL]
IP Addresses	IPv4 dot-notation	[REDACTED_IP]
AWS Keys	AKIA[0-9A-Z]{16}	[REDACTED_SECRET]
Slack Tokens	xoxb-[0-9a-zA-Z-]{+}	[REDACTED_SECRET]
API Keys	sk-[a-zA-Z0-9]{20,}	[REDACTED_SECRET]

Rate Limiting

Rate limiting is configured in `security.rate_limit` with `enabled=true` and `requests_per_minute=60` as default. This is defined in `config.yaml` and intended for gateway-level enforcement.

I. Browser UI

Version 1.2.0 adds two browser-based tools served via the `StaticFiles` mount on the gateway. Both are self-contained HTML files with embedded CSS and JavaScript.

`terminal.html` - Soul Console (29 KB)

The primary browser interface for PsyClaw. Provides a terminal-style UI with: query input targeting the `/query` endpoint, real-time display of answer + sources + retrieval metadata, and a Soul Console panel for managing the identity layer (load current soul, propose evolution, apply evolution with reason, reload from disk). All soul operations use the gateway's REST endpoints.

`extractor.html` - Corpus Extractor (18 KB)

A utility tool for extracting and preparing corpus documents for ingestion into the retrieval pipeline. Carried over unchanged from the original PsyClaw build.

J. Test Suite

PsyClaw includes 8 pytest test files and a PowerShell API test script. All tests use mocked external dependencies (LLM services, embedding model, retriever) - no live services required. The `conftest.py` provides shared fixtures including a complete `TEST_CONFIG` dict that mirrors production config structure.

Test File	Coverage
<code>tests/conftest.py</code>	Shared fixtures: mock LLM, embeddings, retriever, test config dict
<code>tests/test_graph.py</code>	All 7 LangGraph paths: local, grok, offline, gate, error routing
<code>tests/test_gate.py</code>	Gateway endpoints: <code>/query</code> , <code>/health</code> , <code>/soul/*</code> , error responses
<code>tests/test_personality.py</code>	PersonalityManager lifecycle: init, propose, apply, reload, drift
<code>tests/test_sanitizer.py</code>	Prompt filter: banned patterns, max length, corpus sanitization
<code>tests/test_audit.py</code>	Audit logger: SHA-256 hashing, PII redaction, JSONL output
<code>tests/test_hybrid_search.py</code>	HybridRetriever: semantic, keyword, fusion, degradation
<code>tests/test_stemmer.py</code>	Enhanced Porter stemmer: plurals, verb forms, technical terms
<code>tests/apipsTest.ps1</code>	PowerShell: API endpoint smoke tests

K. Deployment

PsyClaw includes a comprehensive `DEPLOYMENT_CHECKLIST.md` (6.5 KB) that covers the full deployment lifecycle for the merged soul-integration build. Key sections:

- Pre-flight: backup host, confirm runtime, verify service assumptions
- Staging: unzip to separate directory, preserve local data (corpus, indices, logs)
- Config review: verify `retrieval.min_score`, `personality.enabled`, `app.mode`
- Validation sequence: `/health` -> `/soul` -> normal query -> soul propose -> apply -> reload -> UI
- Rollback plan: stop gateway, restore backup directory, restart old gateway
- Security notes: cite ID-RAG (arXiv:2509.25299) and MemoryGraft (arXiv:2512.16962)
- Recommended defaults: offline mode, Grok disabled, `min_score` 0.030, manual soul review

Dependencies (requirements.txt)

Package	Version	Purpose
---------	---------	---------

fastapi	0.110.0	HTTP/MCP gateway framework
uvicorn[standard]	0.27.1	ASGI server
pydantic	2.6.1	Request/response validation
langgraph	0.2.60	7-node graph controller
langchain-core	0.3.30	LangGraph dependency
chromadb	0.4.22	Vector store (cosine HNSW)
sentence-transformers	2.5.1	CPU embeddings (384-dim)
rank-bm25	0.2.2	BM25Okapi keyword search
httpx	0.26.0	HTTP client for LLM APIs
pyyaml	6.0.1	Config parsing
numpy	1.26.4	Numeric operations
pytest	8.0.2	Test framework
matplotlib	3.8.3	Optional: metrics visualization

L. Repository Structure

PsyClaw/	
gate.py	FastAPI gateway v1.2.0 (:8787, CORS, static)
graph.py	LangGraph 7-node controller + soul injection
mcp_hybrid_server.py	MCP retrieval server (stdio, JSON-RPC)
metrics.py	Audit log analysis + performance metrics
config.yaml	YAML-driven policy config (all sections)
terminal.html	Browser UI with Soul Console (29 KB)
extractor.html	Corpus extraction tool (18 KB)
requirements.txt	Python dependencies (13 packages)
DEPLOYMENT_CHECKLIST.md	Deployment guide + rollback plan (6.5 KB)
llm/	
client.py	LM Studio + Grok client (typed exceptions)
retrieval/	
indexer.py	Batch corpus indexer (chunk + sanitize + embed)
hybrid_search.py	HybridRetriever (Chroma + BM25 + RRF)
embeddings.py	sentence-transformers (thread-safe singleton)
stemmer.py	Enhanced Porter stemmer (tech-vocab safe)
utils/	
personality.py	Soul governance (SQLite + MD, threading.Lock)
sanitizer.py	Input sanitization + banned patterns
logger.py	Audit logger (SHA-256 + PII redact)
health.py	Health checks (LM Studio, Grok, embeddings)
errors.py	Typed exception hierarchy (7 error types)
schemas/	
api.py	Pydantic models (QueryRequest, SourceInfo, SoulEvolutionRequest)
data/	
personality/	
soul.md	Source-of-truth identity file (1.7 KB)
psyclaw_soul.db	SQLite shadow DB (versions + interactions)
corpus/	
index/	
chroma_db/	ChromaDB vector store (~51 MB, 6 collections)
bm25.pkl	BM25 keyword index (~13 MB)
static/	
terminal.html	Served via StaticFiles mount
logs/	

<code>audit.jsonl</code>	Append-only audit trail
<code>reference/</code>	
<code>original_pre_soul_snapshot/</code>	Pre-soul integration backup
<code>tests/</code>	
<code>conftest.py</code>	Shared fixtures + TEST_CONFIG
<code>test_graph.py</code>	LangGraph path tests (all 7 nodes)
<code>test_gate.py</code>	Gateway endpoint tests
<code>test_personality.py</code>	PersonalityManager lifecycle tests
<code>test_hybrid_search.py</code>	Retrieval + RRF fusion tests
<code>test_sanitizer.py</code>	Input filter + corpus sanitization tests
<code>test_audit.py</code>	Audit logging + PII redaction tests
<code>test_stemmer.py</code>	Stemmer unit tests
<code>apipsTest.ps1</code>	PowerShell API smoke tests

M. Security Invariants

Five invariants are enforced by graph topology and code structure, not by prompts. These cannot be overridden by user input, retrieved content, or LLM output.

1

RAG-First

Retrieval ALWAYS executes before any LLM call. The retrieve node is the mandatory first step in every graph execution. No prompt, configuration, or runtime condition can bypass it.

ENFORCEMENT: Graph edge: ENTRY -> retrieve (unconditional)

2

Topology = Policy

Routing decisions (which LLM to call, whether to go online) are made by graph edges based on score thresholds and config flags - not by prompt instructions. An injected prompt cannot add a new edge to the graph.

ENFORCEMENT: Graph structure: route_score is a conditional edge, not an LLM decision

3

Triple-Gated External Access

The Grok fallback (only external LLM) requires three independent gates: config mode, config flag, and per-query user confirmation. Default state blocks all external calls.

ENFORCEMENT: Code: grok_fallback node only reachable via user_gate conditional

4

Audit Convergence

Every execution path - local_llm, grok_fallback, offline_best - terminates at the audit_logger node. There is no path from any responding node to END that bypasses audit. Personality interactions are also recorded.

ENFORCEMENT: Graph edge: all response nodes -> audit_logger -> END

5

Soul Governance

Soul evolution requires an explicit human reason string via POST /soul/apply. The propose endpoint previews changes without applying. Silent rewrites are structurally impossible. Crash recovery preserves audit integrity via write-order invariant and SHA-256 drift detection.

ENFORCEMENT: Code: apply_evolution() requires reason param; threading.Lock serializes writes